

AI Drive-Thru — Raspberry Pi 5 Setup & Feature Guide

A fully local, single-Pi voice-ordering drive-thru: a camera + mic at the speaker post, a speech pipeline that turns spoken orders into structured JSON (Whisper → DeepSeek → TTS), and a live 3-page web dashboard for kitchen staff. Everything runs on a Raspberry Pi 5 and serves over local WiFi — no cloud, no auth, no payment.



Features

- **Live queue dashboard** (3 pages, no build step): orders appear instantly via WebSocket; staff move them `pending` → `preparing` → `ready` → `picked_up`.
- **Menu editor** — the single source of truth the voice pipeline reads from.
- **Stats / analytics** — orders & revenue today, top items, orders per hour (pure-canvas charts, no chart library).
- **Voice ordering** — mic → Whisper → DeepSeek → TTS, with **order confirmation** (“your order is X — is that correct?”) before anything is submitted.
- **Order photos** — a clean still is captured after the order is parsed (not mid-sentence); staff visually match the customer to the photo.
- **Four run modes** (same code, no separate builds):

Mode	How it triggers	Mic behaviour	Photo
<code>camera</code> (Pi)	YOLOv8n person detection	on only after a person is seen	yes
<code>wakeword</code>	wake word (e.g. “Friday”)	stays on, activates on the word	no
<code>manual</code>	press Enter	on for a fixed window	no
<code>auto</code>	camera if present, else wake word	—	—

- **English output** — spoken prompts and confirmations are English (STT still auto-detects the language; native-language TTS planned for later).
- **South Indian menu** pre-loaded (dosa, idli, vada, pongal, filter coffee, ...).
- **0.96" OLED display** (optional) — shows the order and asks "correct?" on screen.
- **Headless / enclosure ready** — systemd auto-start with crash recovery and offline-safe boot.

Project layout

```
ai_drive_thru/
├── voice_pipeline/           # mic -> Whisper -> DeepSeek -> TTS (separate process)
│   ├── main.py              # orchestration loop (4 modes, confirmation, OLED)
│   ├── test_order.py        # camera-free mic->order test (PC dev / quick check)
│   ├── camera.py            # PresenceDetector (picamera2/cv2) + ManualDetector
│   ├── wake_word.py         # WakeWordDetector (no-camera "Friday" trigger)
│   ├── mic_capture.py       # record while the customer is present
│   ├── stt.py               # faster-whisper (multilingual, auto-detects language)
│   ├── deepseek_client.py   # DeepSeek parsing + confirmation + live menu fetch
│   ├── tts_speaker.py       # pyttsx3 / espeak-ng (English output)
│   ├── display.py           # optional 0.96" OLED (SSD1306)
│   ├── menu.json            # fallback menu (used only if the backend is down)
│   ├── .env.example         # template for DEEPSEEK_API_KEY (used by systemd)
│   └── requirements.txt
├── backend/                 # FastAPI + SQLite + WebSocket (serves the dashboard)
├── dashboard/               # plain HTML/CSS/JS (Live Queue, Menu, Stats)
├── systemd/                 # auto-start units for headless/enclosure use
├── docs/                    # PDF guide
├── data/photos/             # order photos + SQLite DB (created at runtime,
gitignored)
├── run.sh                   # starts the backend (+ dashboard) on :8000
└── README.md
```

Raspberry Pi 5 setup (fresh Raspberry Pi OS, 64-bit Bookworm)

1. Enable the camera and (for the OLED) I2C

```
sudo raspi-config      # Interface Options → Camera → Enable
                      # Interface Options → I2C      → Enable
sudo reboot
```

2. Install apt packages

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y \
  python3-venv python3-pip python3-dev \
  portaudio19-dev libportaudio2 libasound2-dev \
  espeak-ng espeak-ng-espeak \
```

```
libgl1 \
ffmpeg
```

Package	Why
python3-venv , python3-pip , python3-dev	virtualenv + build tools
portaudio19-dev , libportaudio2 , libasound2-dev	sounddevice (mic capture)
espeak-ng , espeak-ng-espeak	TTS — the latter provides the /usr/bin/espeak binary pytttsx3 calls
libgl1	lets opencv-python's cv2 import (YOLO needs it)
ffmpeg	optional; only needed if you swap faster-whisper for openai-whisper

`picamera2 / libcamera` ships with the full image; on the **Lite** image add `libcamera-apps python3-picamera2`.

3. Get the code and install

```
git clone https://github.com/naveenkumar-sudoai/AI-Drive_Thru.git ai_drive_thru
cd ai_drive_thru

# backend + dashboard
chmod +x run.sh
./run.sh # creates .venv, installs deps, serves :8000

# voice pipeline (separate venv - pulls in PyTorch)
cd voice_pipeline
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# (optional) OLED display
pip install luma.oled

# API key
cp .env.example .env
nano .env # set DEEPSEEK_API_KEY=sk-...
```

First run `downloads_yolov8n.pt` (~6 MB) and the Whisper model (~74 MB for `base`) — do this **while online**, before sealing the enclosure.

Running

Backend + dashboard

```
./run.sh # → http://<pi-ip>:8000
```

Voice pipeline

```
cd voice_pipeline && source .venv/bin/activate
export DEEPSEEK_API_KEY="$(cat ../api_key.txt)" # or: source .env
python main.py
```

Choose a mode with `PIPELINE_MODE` :

PIPELINE_MODE	Behaviour
(unset / auto)	camera if present, otherwise wake word (interactive)
camera	force presence-gated (Pi)
wakeword	mic stays on, triggers on <code>WAKE_WORD</code> (default "friday")
manual	press Enter per order

Other env knobs: `WAKE_WORD` , `WHISPER_MODEL` (default `base`), `WHISPER_LANGUAGE` (e.g. `ta`), `BACKEND_URL` , `RECORD_SECONDS` , `MAX_CLARIFICATIONS` , `MAX_CONFIRMATIONS` .

PC test (no camera, no speaker loop)

```
python voice_pipeline/test_order.py # records once, reads back, asks to confirm
```

How an order flows

1. **Trigger** — person seen ~1s (camera), wake word heard (wakeword), or Enter (manual).
2. **Record** — until the person leaves / a cap / a fixed window.
3. **Transcribe** — Whisper, language auto-detected.
4. **Parse** — DeepSeek maps slang & context to the menu ("ghee roast" → "Ghee Roast Dosa"); if ambiguous it asks a clarifying question.
5. **Confirm** — the order is read back ("your order is 1 Masala Dosa, 1 Filter Coffee. Total 120 rupees — is that correct?") and shown on the OLED. It is **only submitted once the customer says yes**; "no"/changes restart the order.
6. **Photo** — a clean still is captured now (camera mode).
7. **Submit** — `POST /order + /upload_photo` ; the queue updates live.
8. **Confirm aloud** — spoken back in the customer's language, then the loop resets.

OLED display (0.96" SSD1306)

- Wire the OLED to the Pi's I2C pins (SDA → GPIO2/SDA, SCL → GPIO3/SCL, VCC → 3V3, GND → GND). The default I2C address is `0x3C` (change via `Display(address=...)` in `display.py`).
- Enable I2C (raspi-config) and `pip install luma.oled` .

- The screen shows the order + total + “Correct? yes/no” during confirmation. If no OLED is connected, `display.py` is a safe no-op, so the pipeline runs identically without it.

Deploy headless in an enclosure (auto-start, reliable)

```
sudo cp systemd/ai-drive-thru-backend.service /etc/systemd/system/  
sudo cp systemd/ai-drive-thru-voice.service /etc/systemd/system/  
# edit User= / WorkingDirectory= / ExecStart= to match your Pi user + clone path  
sudo systemctl daemon-reload  
sudo systemctl enable --now ai-drive-thru-backend  
sudo systemctl enable --now ai-drive-thru-voice
```

Reliability behaviour:

- **Auto-restart** — both units use `Restart=always` (3 s backoff).
- **Offline-safe boot** — `run.sh` skips reinstalling deps when present.
- **Camera failure** — on a headless Pi the pipeline exits (rather than hanging on a keypress) and `systemd` restarts it, retrying the camera.
- **Logs** — `journalctl -u ai-drive-thru-backend -f / -u ai-drive-thru-voice -f`.

Language support

- **Output** — all spoken prompts, clarifications and confirmations are **English**.
- **Input** — Whisper still auto-detects the customer’s language, so a Tamil or Hindi order is transcribed and parsed correctly; only the spoken reply is English. Native-language TTS will be revisited later.

First-run: your menu

The **Menu Editor** tab is the single source of truth. A South Indian menu is pre-loaded in `voice_pipeline/menu.json` (the offline fallback) and can be seeded with `curl -X POST http://localhost:8000/menu -H 'Content-Type: application/json' -d '{"name":"Masala Dosa","price":90,"available":true}'`.

API reference

Method	Path	Purpose
GET	/menu	list menu items
POST	/menu	add item
PUT	/menu/{id}	update item
DELETE	/menu/{id}	remove item
POST	/order	create order
GET	/orders?status=...	list orders, optional filter
PATCH	/orders/{id}	update status
GET	/stats	totals + top items + orders/hour
POST	/upload_photo/{order_id}	upload a photo
WS	/ws	live order broadcasts
GET	/health	liveness check

Statuses: pending → preparing → ready → picked_up.

Performance flags (note & verify — don't silently change)

- **YOLO on Pi 5** (320×240 @ 0.5 Hz): if slow, export to ONNX/NCNN or drop resolution (256×192) — constructor args in `camera.py`.
- **Whisper** — `WHISPER_MODEL=tiny` is fastest.
- **Wake word** — uses Whisper “tiny” on a rolling window (~every 1.5 s); for a lighter/always-on production wake word, swap `wake_word.py` to OpenWakeWord or Porcupine.

Troubleshooting

- **cv2 import error** → `sudo apt install -y libgl1`.
- **No audio** → `python -c "import sounddevice as sd; print(sd.query_devices())"`.
- **No voice** → `sudo apt install -y espeak-ng espeak-ng-espeak`.
- **OLED blank** → confirm I2C enabled and the address (`i2cdetect -y 1`).
- **Dashboard blank** → run `./run.sh` from the repo root.
- **Photos 404** → ensure `data/photos/` exists (created automatically).

Non-goals

No facial recognition (staff visually match the photo), no payments, no login, no cloud — 100% local network, Pi-hosted.